RL Frontiers: Meta-RL, Multi-task RL, Hierarchical RL, Open Problems

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

KAUST Academy
King Abdullah University of Science and Technology

July 30, 2026

Table of Contents

Where We Left Off: Day 9

- Yesterday put RL in the **real world**: **RLHF** (learn the reward when you can't write it down), **multi-agent RL**, and **robotics**.
- Day 9 closed on a warning we pay off today:
  - **multi-agent** RL = many agents sharing *one* environment (non-stationarity);
  - **multi-task** RL = *one* agent across *many* tasks. *These are not the same thing.*
- It also flagged two live frontiers: **emergent complexity** from self-play, and **verifiable rewards** (RLVR) for reasoning agents.

Every method in Days 1–9 optimised one fixed MDP. Today: what happens when we let go of that assumption?

Today: Four Ways RL Generalises Past One MDP

Here a *task* means one MDP: its own dynamics and reward (a specific maze, a specific robot skill).

- **Meta-RL** — *learning to learn*: adapt to a *new* task from a little experience (context *inferred*).
- **Multi-task RL** — one policy over a *fixed set* of tasks (context *given*); where negative transfer comes from.
- **Hierarchical RL** — reasoning over *skills*, not primitive actions: generalising across *time-scales*.
- **Open problems** — what still breaks, and a reflective close on where the field is going.

The thread: *generalising beyond a single fixed problem* — across tasks, to new tasks, and across time.

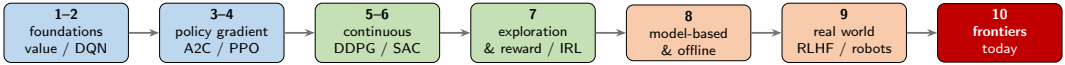
Learning Outcomes

This is an **overview** day. By the end you should be able to:

- **define and place** meta-RL, multi-task RL, and hierarchical RL — and say precisely how they differ;
- explain the core idea of meta-RL — *learning a learner* f_θ — and how RNNs and transformers (*in-context RL*) implement it;
- explain **why** each frontier is hard (exploration as a learned skill, negative transfer, discovering abstractions);
- name **representative methods** (RL^2 / Decision Transformer, PCGrad, the options framework) and articulate the main **open problems**, linking back to Days 1–9.

Goal: to recognise and situate these frontiers, not to implement each from scratch.

The Whole Journey: Ten Days



From *how to optimise one MDP* (Days 1–6) → *where the reward and data come from* (Days 7–8) → *the real world* (Day 9) → *past a single MDP entirely* (today).

Transfer Learning and **Meta-Learning**

What's the problem?

this is easy (mostly)

this is difficult

Why?

Can RL use the same prior knowledge as us?

- If we've solved prior tasks, we might acquire useful knowledge for solving a new task
- How is the knowledge stored?
  - **Q-function:** tells us which actions or states are good
  - **Policy:** tells us which actions are potentially useful
    - some actions are never useful!
  - **Models:** what are the laws of physics that govern the world?
  - **Learned features:** the patterns a network extracts from raw input — e.g. a DQN playing from pixels (Day 2) builds early-layer detectors for the ball & paddle; that "vision" can transfer to a new game even when the policy cannot
    - Don't underestimate this!

How can we frame transfer learning problems?

Transfer learning: carry knowledge from prior (*source*) tasks to a new (*target*) task, so you don't start from scratch. Three ways to set it up:

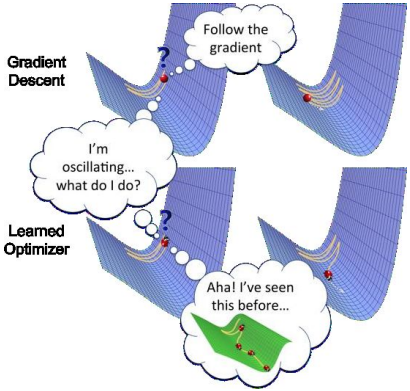
1. **Forward transfer:** learn policies that transfer effectively
   a) Train on source task, then run on target task (or finetune)
   b) Relies on the tasks being quite similar!
2. **Multi-task transfer:** train on many tasks, transfer to a new task
   a) Sharing representations and layers across tasks in multi-task learning
   b) New task needs to be similar to the distribution of training tasks
3. **Meta-learning:** learn to learn on many tasks
   a) Accounts for the fact that we'll be adapting to a new task during training!

What issues are we likely to face?

- **Domain shift:** representations learned in the source domain might not work well in the target domain
- **Difference in the MDP:** some things that are possible to do in the source domain are not possible to do in the target domain
- **Finetuning is hard:** adapting to the new task still needs *exploration*, but a well-pretrained policy is confident and nearly *deterministic* — so it barely explores. The sharpness that made pretraining good now works against us.

What is meta-learning?

- **Meta-learning = learning to learn:** use *many* training tasks to learn a *learning procedure* that adapts to a *new* task fast.
- Having already solved 100 tasks isn't a burden — it's the *signal* we learn from. (Closely related to multi-task learning.)
- **What could "the learner" be?** Three recipes for the same goal:
  - a **learned optimizer** — replace hand-tuned SGD with a *learned* update rule (the figure);
  - a **sequence model (RNN)** that reads past experience and adapts with *no extra gradient steps* — *the route we follow today*;
  - a **representation** that a few fine-tuning steps adapt to any task (e.g. MAML).



A *learned optimizer*: having seen many loss landscapes, it recognises "I've seen this before" and navigates a new one better than plain gradient descent.

Why is meta-learning a good idea?

- Deep reinforcement learning, especially model-free, requires a huge number of samples
- If we can meta-learn a faster reinforcement learner, we can learn new tasks efficiently!
- What can a meta-learned learner do differently?
  - Explore more intelligently
  - Avoid trying actions that are know to be useless
  - Acquire the right features more quickly

Warm-up: "learning to learn" in supervised learning

The cleanest place to see the idea — **few-shot image classification**:

- you are shown a handful of labelled images of classes you have *never seen* — the *support set* (a tiny training set $\mathcal{D}^{\text{tr}}$);
- then you must label a *new* image — the *query* x.

Ordinary learning trains *once* on one big dataset. Here each **task** gives only a *few* examples — but there are **many** tasks to learn the *trick* from.

one concrete task — classes "A" vs "B":

support set $\mathcal{D}^{\text{tr}}$ (you are shown):

a *labelled example* (x, y): image $x \rightarrow$ label y

query x (new, unlabelled):

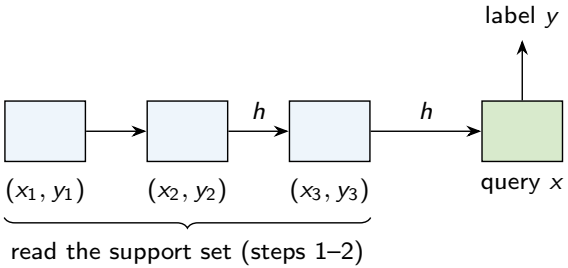
$$\underbrace{f(\mathcal{D}^{\text{tr}},\ x)}_{\text{read support set, then classify } x} \rightarrow y$$

How? A sequence model reads the support set

One network, three steps:

1. feed the labelled examples $(x_1, y_1), (x_2, y_2), \ldots$ into an **RNN**, one at a time;
2. its **hidden state** h builds a *summary* of the support set — what each class looks like;
3. feed the **query** x; conditioned on h, the RNN outputs the label y.

Why an RNN? it can swallow *any number* of examples and condition its answer on all of them — *no separate training run, no gradient steps at test time*.



the green cell (step 3) is where it actually predicts — after "reading" everything before it.

What is actually learned? Two kinds of parameters

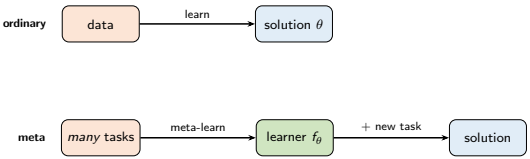
The network carries **two** kinds of parameters:

- *meta-weights* θ — the RNN's weights, **shared across all tasks** and *trained* so the RNN behaves like a good learning algorithm. *Slow*, learned once (over thousands of tasks).
- *hidden state* h_i — formed **per task** by reading that task's support set. *Fast*, no gradient, different every task.

$$\phi_i = [\ \underbrace{h_i}_{\text{fast, per-task}}\ ,\ \underbrace{\theta_p}_{\text{slow, shared}}\ ]$$

Training = show the RNN *thousands* of tasks; tune θ so that, after reading each support set, it predicts the query well.

The big idea: learn a *learner*, not a solution



Normally we learn a **solution**: data → weights that solve *this* task — then throw them away for the next one.

Meta-learning learns a learner f_θ: a function that turns a new task's experience *into* a good solution. **That** is what we keep.

Everything ahead just *builds* f_θ:

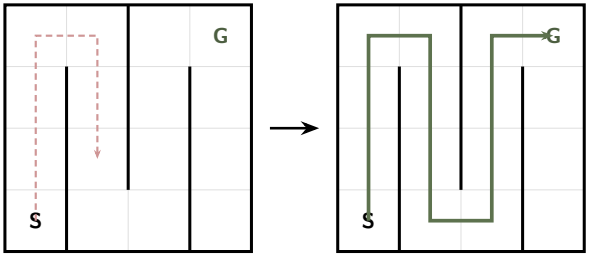
- an RNN reading a support set (just now);
- an RNN reading experience in an MDP (next);
- a transformer reading a history (in-context RL, later).

We don't learn the answer — we learn *how to produce answers*.

Meta **Reinforcement Learning**

Meta-RL Example: Maze Navigation

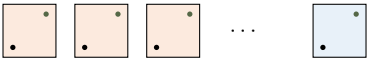
Same shape as few-shot (slides 14–16) — only now the agent must first **earn** *its own training set by exploring, then use it to solve.*



explore
collect $\mathcal{D}_{\text{tr}} \sim \pi^{\text{exp}}$

solve
$\mathcal{D}_{\text{tr}} \to \pi^{\text{task}}$

Meta-train: learn an explorer π^{exp} and a solver π^{task} over *many* mazes.



Meta-test: a *new* maze — explore a little, then solve.

Key assumption: train & test mazes are drawn from the *same distribution* — otherwise there is no reason adaptation should generalize.

Maze meta-RL setup: Duan et al. '17.

The meta-RL problem: an analogy

"Generic" learning:

$$\theta^\star = \arg\min_\theta \mathcal{L}(\theta, \mathcal{D}^{\text{tr}}) = f_{\text{learn}}(\mathcal{D}^{\text{tr}})$$

Reinforcement learning:

$$\theta^\star = \arg\max_\theta \mathbb{E}_{\pi_\theta}[R(\tau)] = f_{\text{RL}}(\mathcal{M})$$

RL is itself a function: an MDP $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, r\}$ goes *in*, an optimal policy comes *out*.

"Generic" meta-learning:

$$\theta^\star = \arg\min_\theta \sum_i \mathcal{L}\left(\phi_i, \mathcal{D}_i^{\text{ts}}\right), \quad \phi_i = f_\theta(\mathcal{D}_i^{\text{tr}})$$

Meta-reinforcement learning:

$$\theta^\star = \arg\max_\theta \sum_i \mathbb{E}_{\pi_{\phi_i}}[R(\tau)], \quad \phi_i = f_\theta(\mathcal{M}_i)$$

learn the adapter f_θ: it turns an MDP $\mathcal{M}_i$ into a good policy, and we train it across *many* MDPs.

f_θ is the *meta-learner*: it maps a *task* $\mathcal{M}_i \mapsto$ *policy parameters* ϕ_i; we train θ so it does this well across many tasks.

(The parallel: the dataset $\mathcal{D}^{\text{tr}}$ becomes an MDP $\mathcal{M}$; "adapt, then score" is the same recipe — the loss just becomes a return.)

The meta-RL problem: a distribution of tasks

$$\theta^\star = \arg\max_\theta \sum_i \mathbb{E}_{\pi_{\phi_i}}[R(\tau)], \quad \phi_i = f_\theta(\mathcal{M}_i)$$

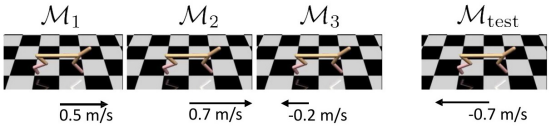
assumption: $\mathcal{M}_i \sim p(\mathcal{M})$ — tasks are *drawn from a distribution*; train and test tasks must *share* it, or there is nothing to generalise from.

meta-test (the real exam): drop the trained learner into a *brand-new* task $\mathcal{M}_{\text{test}} \sim p(\mathcal{M})$; it must adapt, $\phi = f_\theta(\mathcal{M}_{\text{test}})$, from a little experience.

$\{\mathcal{M}_1, \ldots, \mathcal{M}_n\}$

meta-training MDPs

here $p(\mathcal{M})$ = a distribution over *target running speeds*:



each task rewards a different velocity; a new task is just a new target speed.

That pins down the **problem**. Building the adapter f_θ now takes three steps: *(1)* which task am I in? *(2)* how to implement it? *(3)* why does it explore?

Building the meta-learner (1/3): which task am I in?

$$\theta^\star = \arg\max_\theta \sum_i \mathbb{E}_{\pi_{\phi_i}}[R(\tau)]$$

where $\phi_i = f_\theta(\mathcal{M}_i)$

$\Rightarrow$

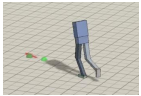
feed the whole history as *context*:

$$\pi_\theta\big(a_t \mid s_t,\ \underbrace{s_1, a_1, r_1, \ldots, s_{t-1}, a_{t-1}, r_{t-1}}_{\text{history so far} = \text{context}}\big)$$

how we use it: the policy reads its *own* past experience. With an *empty* history it must *explore*; with a *rich* one it can *exploit* — same weights θ, behaviour driven by the context.

meta-RL: context is *inferred* from experience. *multi-task RL:* context ϕ_i is *given*. (how to *carry* a growing history? $\rightarrow$ an RNN, **next**.)

ϕ: stack location

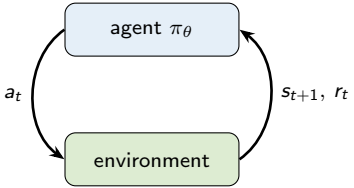


ϕ: walking direction

ϕ: where to hit puck

the photos show *what varies across tasks* — what the context ϕ must encode

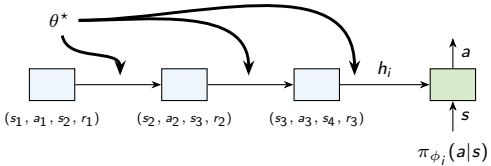
Building the meta-learner (2/3): use a recurrent policy



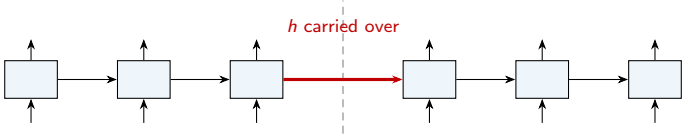
so... we just train an **RNN policy**? *yes* — the recurrent state *is* the adapter, $\phi_i = [\, h_i,\ \theta_\pi \,]$.

$f_\theta(\mathcal{M}_i)$ must do two jobs:

1. **improve** the policy from experience $\{(s_1, a_1, s_2, r_1), \dots\}$;
2. **choose how to interact** — so meta-RL must also *choose how to explore*.



crucially, the RNN hidden state is not reset between episodes:



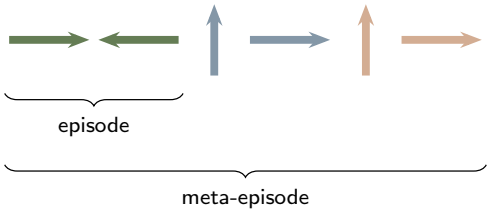
Building the meta-learner (3/3): why it explores

The RNN is trained to maximise reward over the **whole meta-episode** — many episodes back-to-back, hidden state never reset:

$$\theta^\star = \arg\max_\theta \mathbb{E}_{\pi_\theta}\Big[\underbrace{\sum_{t=0}^{T} r(s_t, a_t)}_{\text{summed across } \textit{all} \text{ episodes}} \Big]$$

So *spending early steps exploring pays off*: what the agent learns now earns more reward later in the *same* meta-episode. Exploration becomes *optimal behaviour*, not a bolt-on.

one meta-episode = several episodes:



Contrast Day 7: there exploration was a bolt-on *bonus* (count-based, RND); here the meta-learner *learns* an exploration strategy end-to-end.

Black-Box Meta-RL: Algorithm

Meta-Training

1. Sample task $\mathcal{T}_i$
2. Roll-out policy $\pi(a \mid s, \mathcal{D}_i^{\text{tr}})$ for N episodes (under dynamics $p_i(s' \mid s, a)$ and reward $r_i(s, a)$)
3. Store sequence in replay buffer for task $\mathcal{T}_i$
4. Update policy to maximize discounted return for all tasks.
5. **Repeat** (go back to step 1).

Meta-Test Time

1. Sample new task $\mathcal{T}_j$
2. Roll-out policy $\pi(a \mid s, \mathcal{D}_j^{\text{tr}})$ for up to N episodes

Meta-RL Example

Experiment: Learning to visually navigate a maze — train on 1000 small mazes, test on held-out small mazes and large mazes

Method	Small Maze		Large Maze	
	Episode 1	Episode 2	Episode 1	Episode 2
Random	188.6 ± 3.5	187.7 ± 3.5	420.2 ± 1.2	420.8 ± 1.2
LSTM	52.4 ± 1.3	39.1 ± 0.9	180.1 ± 6.0	150.6 ± 5.9
SNAIL (ours)	$\mathbf{50.3 \pm 0.3}$	$\mathbf{34.8 \pm 0.2}$	$\mathbf{140.5 \pm 4.2}$	$\mathbf{105.9 \pm 2.4}$

From: Mishra, Rohaninejad, Chen, Abbeel. A Simple Neural Attentive Meta-Learner. ICLR 2018

Architectures for meta-RL

All implement the same $f_\theta(\mathcal{M}_i)$: read a history of experience $\rightarrow$ produce context ϕ_i. They differ in *how they read it*.

Recurrent (RL^2)

Roll history through an **RNN**; hidden state h_i *is* the context.

+ simple, general
− long-horizon memory is hard

Attentive (SNAIL)

Temporal convolutions + **attention** over the whole history.

+ pinpoints relevant past steps
− heavier, context-window bound

Context encoder (PEARL)

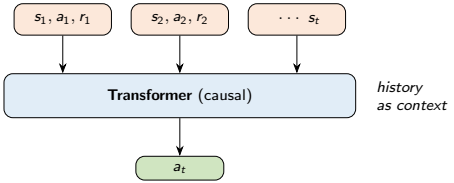
Encode experience into a **latent** z (*permutation-invariant*); condition the policy on z.

+ off-policy, sample-efficient
− decouples explore vs. solve

Duan et al. RL^2. 2016. • Wang et al. Learning to Reinforcement Learn. 2016. • Mishra et al. SNAIL (A Simple Neural Attentive Meta-Learner). ICLR 2018. • Rakelly et al. PEARL (Efficient Off-Policy Meta-RL via Probabilistic Context Variables). ICML 2019.

Meta-RL, 2020s: just swap the RNN for a transformer

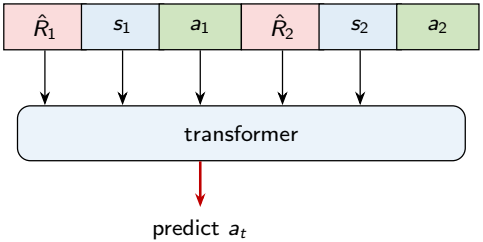
- Recall the crucial point: the RNN hidden state is **not reset between episodes** — the policy reads its *whole* interaction history.
- That is exactly *in-context learning*. Replace the RNN with a **transformer** and the history becomes the **context window**.
- At meta-test time: **no gradient updates.** The model adapts by *reading* more experience, like few-shot prompting in an LLM.
- Same objective as before ($\phi_i = f_\theta(\mathcal{M}_i)$) — only the architecture of f_θ changed.



the hidden state became a context window

Decision Transformer: RL as conditional sequence modelling

- **Idea:** treat a trajectory like a *sentence*. Its "tokens" are $(\hat{R}_t, s_t, a_t)$ in time order; train a transformer to **predict the next action** — an ordinary next-token loss. No Bellman, no policy gradient.
- $\hat{R}_t$ = **return-to-go**: how much reward you still want from here on.
- **Use it:** at test time *prompt* it with a big desired return $\hat{R}$ and the current state; it outputs the action that tends to reach that return.
- Learns from a **fixed dataset** (offline, Day 8) — it just imitates, *conditioned on the outcome*.
- *Intuition:* like an LLM predicting the next *word* — here it predicts the next *action*, told "how well I want to do".

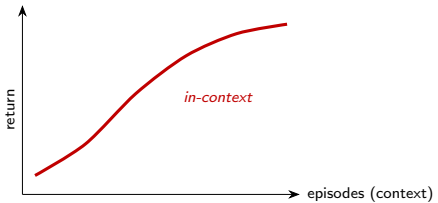


condition on the return you want

Chen et al. Decision Transformer: RL via Sequence Modeling. NeurIPS 2021.

Algorithm Distillation: learning to learn, in-context

- Decision Transformer copies *good behaviour*; Algorithm Distillation copies the **act of getting better**.
- **Recipe:** run an ordinary RL agent on many tasks and *log its whole training* — clumsy early episodes through skilled late ones. Train a transformer to predict *"what did it do next?"* across that log.
- So the transformer soaks up the *improvement itself*, not any one policy.
- **Payoff:** on a *new* task, as its context fills with episodes it **keeps getting better on its own** — same curve, *no weight updates*.
- The cleanest "*learning to learn = in-context learning*" — the same few-shot mechanism students see in LLMs.



no gradient steps —
the curve is produced by reading experience

Laskin et al. In-context RL with Algorithm Distillation. ICLR 2023.

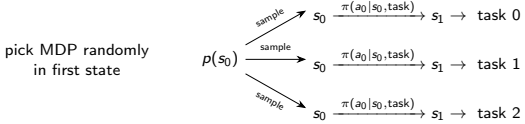
Multi-Task **Reinforcement Learning**

What is multi-task RL?

- One policy that solves a **fixed set** of tasks — the task identity is usually *given* as part of the input.
- Formally: fold the task into the state. Sample an MDP at the start, then act:

$$\pi_\theta(a_t \mid s_t, \text{task})$$

- Contrast **meta-RL** (just now): there the context ϕ_i is *inferred* from experience, because the task is *new*. Here it is handed to you.
- Why bother? *shared structure* — one network reuses skills and representations across tasks.



one policy, many tasks, task-ID conditioned

Do not confuse: multi-*agent* vs multi-*task*

The single most confusable pair in the course — and exactly the distinction Day 9 promised to settle.

Multi-agent RL (Day 9)

- **Many** agents, **one** environment.
- They act at the same time; each one's learning makes the environment *non-stationary* for the others.
- Core problem: a *moving target*. Fixes: CTDE, self-play.

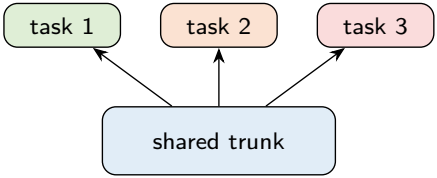
Multi-task RL (today)

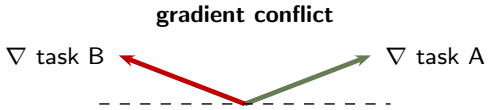
- **One** agent, **many** tasks.
- Tasks are separate MDPs; the agent solves each, reusing *shared structure*.
- Core problem: tasks *competing* over shared parameters (next slide).

Many agents / one world $\neq$ one agent / many worlds.

Architectures — and the central pathology

- Standard recipe: a **shared trunk** plus **per-task heads**, or a goal-/task-conditioned policy $\pi(a \mid s, \text{task})$.
- The hope: tasks *help* each other (positive transfer).
- The reality: *negative transfer / gradient conflict* — task gradients point in *opposing* directions and fight over the shared weights.
- Net effect: joint training can be *worse* than training each task alone.





Methods: making tasks share nicely

- **PCGrad (gradient surgery).** When two task gradients conflict, *project* one onto the normal plane of the other before summing — remove only the fighting component.
- **Distral.** Learn a shared "distilled" policy and regularise each task's policy *toward* it — share behaviour, not raw gradients.
- **Policy distillation** (the general tool): train specialists, then compress them into one multi-task student.
- **Benchmark: Meta-World** — 50 robot-manipulation tasks, the canonical test-bed for *both* multi-task and meta-RL (it bridges back to the last section).

One policy across many tasks is also the seed of the "generalist agent" trend — which turns into a scaling open problem later today.

Yu, Kumar, Gupta, Levine, Hausman, Finn. Gradient Surgery for Multi-Task Learning. NeurIPS 2020.

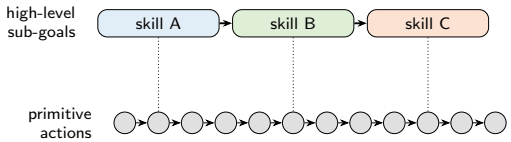
Teh et al. Distral: Robust Multitask Reinforcement Learning. NeurIPS 2017.

Yu, Quillen, He, Julian, Hausman, Finn, Levine. Meta-World. CoRL 2019.

Hierarchical **Reinforcement Learning**

Why hierarchy? Flat RL and the long horizon

- Flat RL chooses a **primitive action every step**. Over a long, sparse-reward task that is a needle in an astronomically large haystack.
- Think **Montezuma's Revenge** (Day 7): thousands of steps before any reward — random exploration essentially never stumbles on it.
- Humans don't plan at the muscle-twitch level. We reason over *skills* ("get the key", "open the door").
- **Temporal abstraction:** a high-level policy picks *sub-goals / skills*; a low-level policy executes them over many steps.



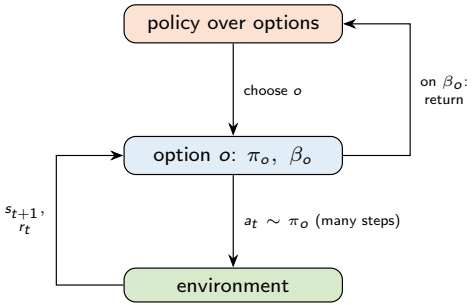
a few decisions instead of hundreds

The classic framework: options

An **option** is a temporally-extended action — a mini-policy with a start and a stop. Formally a triple:

- **Initiation set** $\mathcal{I}$ — where the option can be started;
- **Intra-option policy** $\pi_o(a \mid s)$ — how it acts while running;
- **Termination** $\beta_o(s) \in [0, 1]$ — probability of stopping here.

- The policy-over-options picks an option; it runs for *several* steps; then control returns to the top.
- This turns the MDP into a *semi-MDP* (actions take variable, extended time).



Sutton, Precup, Singh. The options framework. *Artificial Intelligence*, 1999.

Learning the hierarchy, not hand-designing it

HIRO — goals *are* states

- A **manager** outputs a *goal state* g every k steps.
- A **worker** is rewarded for *getting closer to* g (distance in state space).
- Most intuitive form of hierarchy; off-policy, so sample-efficient enough for real robots.

Option-Critic — learn options end-to-end

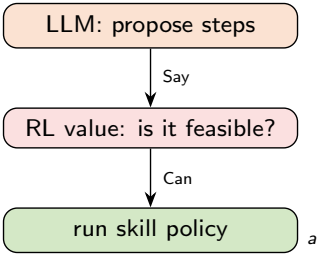
- Learn the intra-option policies *and* their terminations *and* the policy-over-options — all by policy gradient.
- The payoff: *no hand-designed options*; the hierarchy is discovered from reward.
- (FeUdal Networks: same manager/worker idea with goals in a *learned latent* space.)

Manager sets the *goal*; worker earns reward for *reaching* it.

Nachum et al. HIRO. NeurIPS 2018. • Bacon, Harb, Precup. Option-Critic. AAAI 2017.

The 2020s angle: LLMs as the high-level planner

- Why teach hierarchy *after* Day 9? Because the hottest instance is an **LLM planning over learned skills**.
- **SayCan:** the LLM proposes a plausible high-level step ("pick up the sponge"); a learned *value/affordance* function grounds it in what the robot can *actually do* right now.
- *Say* (language prior, what's useful) $\times$ *Can* (RL value, what's feasible) $\rightarrow$ the next skill to run.
- This *is* a hierarchy: language plan on top, RL skill-policies underneath.



[a] Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan). CoRL 2022.

Still open: *automatically discovering good abstractions* remains unsolved — most wins still lean on hand-designed or task-specific hierarchies.

Challenges and **Open Problems** in RL

What's the problem?

Challenges with core algorithms:

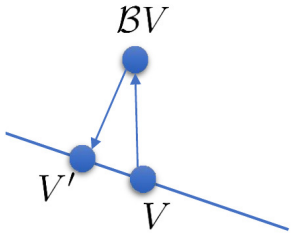
- **Stability:** does your algorithm converge?
- **Efficiency:** how long does it take to converge? (how many samples)
- **Generalization:** after it converges, does it generalize?

Challenges with assumptions:

- Is this even the right problem formulation?
- What is the source of supervision?

Stability and hyperparameter tuning

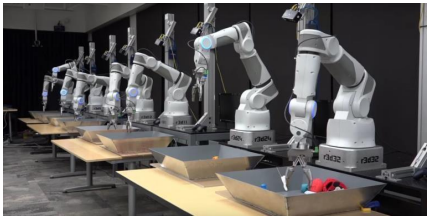
- Devising stable RL algorithms is very hard
- Q-learning / value function estimation
  - Fitted Q / fitted value methods with deep network function estimators are typically not contractions, hence no guarantee of convergence
  - Lots of parameters for stability: target network delay, replay buffer size, clipping, sensitivity to learning rates, etc.
- Policy gradient / likelihood ratio / REINFORCE
  - Very high variance gradient estimator
  - Lots of samples, complex baselines, etc.
  - Parameters: batch size, learning rate, design of baseline
- Model-based RL algorithms
  - Model class and fitting method
  - Optimizing policy w.r.t. model non-trivial due to backpropagation through time
  - More subtle issue: policy tends to exploit the model



The challenge with sample complexity

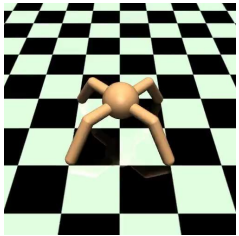
- Need to wait for a long time for your homework to finish running
- Real-world learning becomes difficult or impractical
- Precludes the use of expensive, high-fidelity simulators
- Limits applicability to real-world problems

This thread ran through Days 5–8: off-policy replay, model-based rollouts, and offline data are all responses to sample complexity.



Scaling up deep RL & generalization

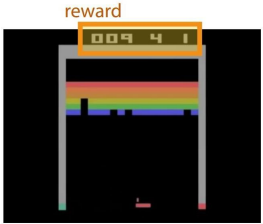
- Large-scale
- Emphasizes diversity
- Evaluated on generalization



- Small-scale
- Emphasizes mastery
- Evaluated on performance
- Where is the generalization?

Where does the supervision come from?

- If you want to learn from many different tasks, you need to get those tasks — and their rewards — from *somewhere*.
- **Learn the reward from demonstrations** — inverse RL (*recall Day 7*: one answer is to *learn* the reward instead of writing it).
- **Generate objectives automatically?**
- **Verifiable rewards (RLVR, Day 9):** when correctness is automatically checkable — math, code, reasoning (e.g. DeepSeek-R1) — the reward needs no human labels at all.



Mnih et al. '15

reinforcement learning agent

what is the reward?

Rethinking the Problem Formulation

- How should we define a control problem?
  - What is the data?
  - What is the goal?
  - What is the supervision? (may not be the same as the goal...)
- Think about the assumptions that fit your problem setting!
- Don't assume that the basic RL problem is set in stone

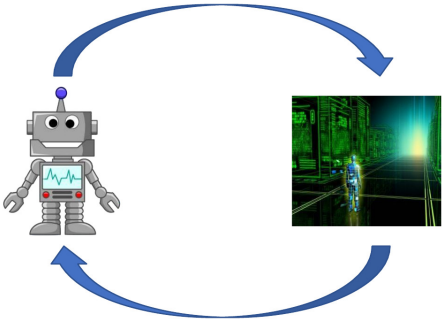
Some perspectives to close on. . .

- Reinforcement Learning as an **Engineering Tool**
- Reinforcement Learning and the **Real World**
- Reinforcement Learning as "**Universal**" Learning

Less about algorithms now — more about what RL is for.

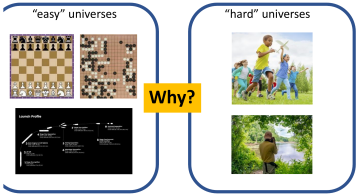
RL as an engineering tool

- Seen from engineering, RL is barely new. Classical control already says: *characterize* the system, *simulate* it, then *control* it.
- RL just swaps the last step: characterize, simulate, **run RL**.
- Its real role is a *powerful inversion engine* — given a simulator and a goal, it searches out the actions that achieve it.
- *Slogan:* **anything you can simulate, you can control.**
- *Main weakness:* you *still need to simulate* — and the real world is hard to simulate.



Moravec's paradox

Moravec's paradox seems like a statement about AI, but it is really a statement about the *physical universe*.



"We are all prodigious olympians in perceptual and motor areas, so good that we make the difficult look easy. Abstract thought, though, is a new trick, perhaps less than 100 thousand years old. We have not yet mastered it. It is not all that intrinsically difficult; it just seems so when we do it." — Hans Moravec

"The main lesson of thirty-five years of AI research is that the hard problems are easy and the easy problems are hard. The mental abilities of a four-year-old that we take for granted — recognizing a face, lifting a pencil, walking across a room, answering a question — in fact solve some of the hardest engineering problems ever conceived." — Steven Pinker

What does this have to do with RL?

How do we engineer a system that can deal with the unexpected?

- Minimal external supervision about what to do
- Unexpected situations that require adaptation
- Must discover solutions autonomously
- Must "stay alive" long enough to discover them!

- Humans are extremely good at this
- Current AI systems are extremely bad at this
- RL in principle can do this, and nothing else can

So what's the problem?

RL in principle can do this, and nothing else can — *but we rarely study this kind of setting in RL research!*

"easy" universes	**"hard" universes**
success = high reward ("optimal control")	success = "survival" ("good enough control")
closed world, rules are known	open world, everything must come from data
lots of simulation	no simulation (because rules are unknown)
Main question: can RL algorithms optimize really well	Main question: can RL generalize and adapt

RL should be really good in the "hard" universes!

This all seems really hard, what's the point?

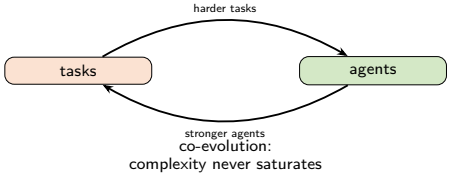
Why is this interesting?

- It's exciting to see what solutions intelligent agents come up with
- Most exciting if they come up with something we don't expect
- This requires the world they inhabit to admit novel solutions
- This means that world must be complicated enough!

To see interesting emergent behavior, we must train our systems in environments that actually require interesting emergent behavior!

Open-endedness: no fixed task distribution at all

- If complexity must *come from the environment*, where does an endless supply of it come from?
- **Self-play (Day 9)** was one engine: agents generate their own *curriculum* automatically, each one's progress raising the bar for the others.
- **Open-endedness / POET:** *co-evolve* the tasks *and* the agents — never a fixed objective, always a new frontier just beyond reach.
- The endpoint of today's thread: meta and multi-task still assumed $\mathcal{M}_i \sim p(\mathcal{M})$; open-endedness gives up even that — *no fixed task distribution at all.*



Wang, Lehman, Clune, Stanley. Paired Open-Ended Trailblazer (POET). 2019.

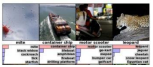
Stepping back: why do we need learning at all?

Why do we need **machine learning** anyway?

A postulate: We need machine learning for one reason and one reason only — that's to produce **adaptable and complex decisions**.

Decision: how do I move my joints?

Decision: how do I steer the car?



What is the decision? The image label?

Usually not!

What happens with that label afterwards? *These are* **decisions** *— they lead to* **consequences**

Aside: why do we need **brains** anyway?

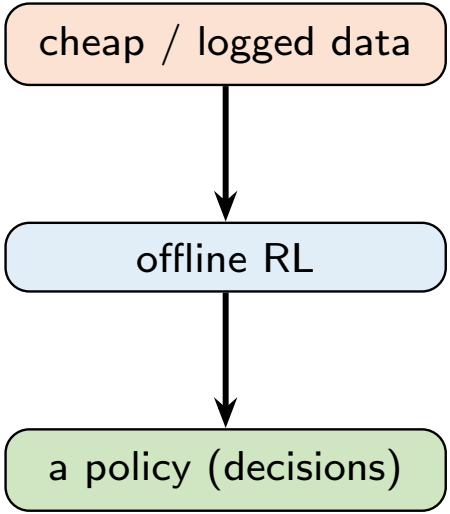
Daniel Wolpert

(knows quite a lot about brains)

"We have a brain for one reason and one reason only — that's to produce **adaptable and complex movements**. *Movement is the only way we have affecting the world around us. . . I believe that to understand movement is to understand the whole brain."*

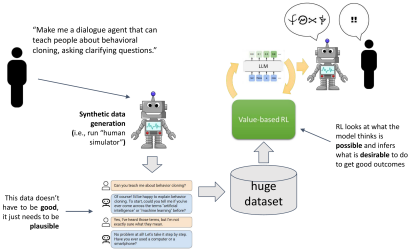
RL as "universal" learning: using cheap data

- The lesson of large-scale ML: progress comes from **diverse data** and *reducing the supervision burden* per example.
- RL's angle on this: it can learn from *cheap, un-curated experience* — not just hand-labelled examples.
- **Offline RL (Day 8):** squeeze a good policy out of a *fixed dataset* of past interaction — no new environment steps, no fresh labels.
- That reframes RL as a general recipe for turning *logged experience* into *decisions* — the same engine, pointed at whatever data is cheap.



A concrete case: offline RL for LLM dialogue

- *Recall Day 8:* offline RL learns from a fixed dataset with no environment interaction (MOReL / MOPO / COMBO — pessimism keeps it honest).
- Point that engine at **multi-turn dialogue**: an LLM is a cheap *simulator of human conversation*.
- Instead of *imitating* human replies, RL optimises for a downstream **outcome** (persuade, book, resolve) using the LLM's model of how people respond.
- RL + LLM shines exactly when a task is *easy to simulate* but *hard to do optimally*.

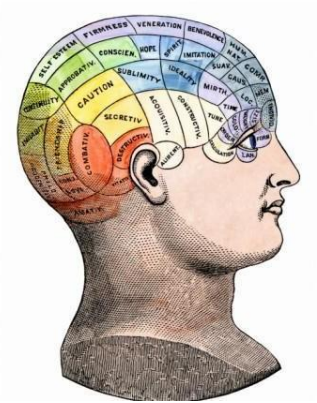


Hong, Levine, Dragan. Goal-Directed Dialogue via RL on Imagined Conversations. 2023.

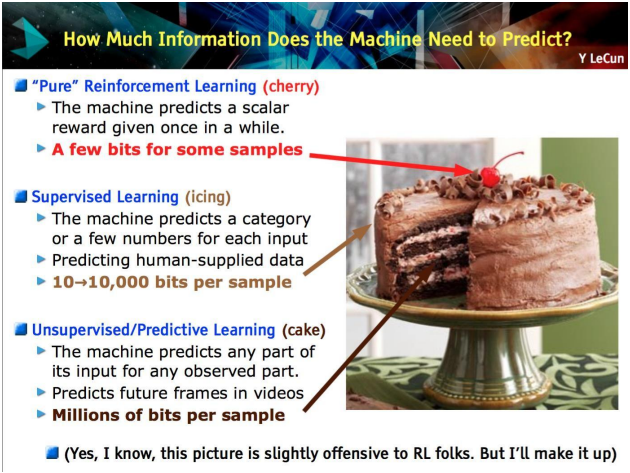
Learning as the basis of intelligence

- Reinforcement learning = can reason about decision making
- Deep models = allows RL algorithms to learn and represent complex input-output mappings

Deep models are what allow reinforcement learning algorithms to solve complex problems end to end!



What is missing?



Today, in one line

Everything today was about RL *generalising beyond one fixed MDP*.

Meta-RL

adapt to a *new* task from a little experience — context *inferred* (RL^2, PEARL, in-context RL).

Multi-task RL

one policy over *many* tasks — context *given*; beware negative transfer (PCGrad, Distral).

Hierarchical RL

reason over *skills* across time-scales (options, HIRO, Option-Critic, SayCan).

Across tasks $\rightarrow$ to new tasks $\rightarrow$ across time. And a field still wide open: stability, sample cost, where supervision comes from, and how abstractions are discovered.

Ten days: from one MDP to the frontier

- You built the machinery (value, policy gradient, actor–critic, continuous control), learned where *reward and data* come from (exploration, IRL, model-based, offline), took it to the *real world* (RLHF, MARL, robotics), and today looked *past a single problem* entirely.
- The recurring lesson: *the algorithm is rarely the hard part* — the problem formulation, the reward, and the data usually are.

Thank you — go build something that uses RL's chaos to somehow reach order.

References

Meta-RL & in-context RL.

- Duan, Schulman, Chen, Bartlett, Sutskever, Abbeel. RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning. 2016. Wang et al. Learning to Reinforcement Learn. 2016.
- Mishra, Rohaninejad, Chen, Abbeel. A Simple Neural Attentive Meta-Learner (SNAIL). ICLR 2018.
- Rakelly, Zhou, Finn, Levine, Quillen. Efficient Off-Policy Meta-RL via Probabilistic Context Variables (PEARL). ICML 2019.
- Chen et al. Decision Transformer: RL via Sequence Modeling. NeurIPS 2021.
- Laskin et al. In-context Reinforcement Learning with Algorithm Distillation. ICLR 2023.

Multi-task RL.

- Teh et al. Distral: Robust Multitask Reinforcement Learning. NeurIPS 2017.
- Yu, Kumar, Gupta, Levine, Hausman, Finn. Gradient Surgery for Multi-Task Learning (PCGrad). NeurIPS 2020.
- Yu et al. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta RL. CoRL 2019.

Hierarchical RL.

References (cont.)

- Sutton, Precup, Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction (options). *Artificial Intelligence*, 1999.
- Bacon, Harb, Precup. The Option-Critic Architecture. AAAI 2017. Vezhnevets et al. FeUdal Networks for Hierarchical RL. ICML 2017.
- Nachum, Gu, Lee, Levine. Data-Efficient Hierarchical RL (HIRO). NeurIPS 2018.
- Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan). CoRL 2022.

References (cont.)

Open problems & reflection.

- Christiano et al. Deep RL from Human Preferences. NeurIPS 2017.
- Hong, Levine, Dragan. Zero-Shot Goal-Directed Dialogue via RL on Imagined Conversations. 2023.
- Wang, Lehman, Clune, Stanley. Paired Open-Ended Trailblazer (POET). 2019.

These slides have been adapted from

- Chelsea Finn, CS224R: Deep Reinforcement Learning, Stanford
- Sergey Levine, CS285: Deep Reinforcement Learning, Berkeley